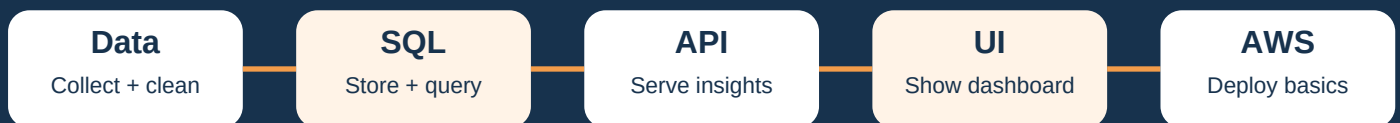


Smart Urban Mobility Analytics Dashboard



21-Day guided project handbook for Ashish

Designed as a teacher-style workflow manual: data analysis + SQL + GitHub + backend + frontend + AWS



What this handbook gives Ashish

A complete 21-day learning path with daily goals, exact workflow, why each task matters, \nwhat files to create, wh

How to use this handbook

This project is intentionally designed as an **uncommon analytics topic** so Ashish builds something stronger than a normal sales dashboard. The topic is urban mobility: trip volume, traffic patterns, route activity, peak hours, and time-based transport insights. The handbook is written like a teacher, not a checklist. That means each day explains **what to do**, **how to do it**, **why it matters**, and **what should exist before moving to the next day**.

The goal is not to finish by rushing. The goal is to finish with a clean end-to-end system that Ashish can explain confidently in interviews: dataset -> cleaning -> Python analysis -> SQL database -> backend APIs -> frontend dashboard -> AWS deployment basics.

Recommended dataset type

Use a transportation dataset such as taxi trips, ride-sharing trips, bike-sharing trips, public transport logs, or city traffic sensor data. A small or medium subset is best for learning because it keeps the work manageable while still allowing meaningful analysis.

Recommended tech stack

Python with Pandas for cleaning and analysis; MySQL or PostgreSQL for SQL; GitHub for version control; Flask for the backend API; HTML/CSS/JavaScript with Chart.js for the frontend; AWS S3 for static frontend hosting; EC2 only if Ashish is ready for a beginner deployment experiment.

Full workflow map

Stage	What Ashish does	Why this matters
Data prep	Choose one dataset, understand columns, clean inconsistencies, and engineer useful new fields such as hour_of_day or trip_duration.	Without reliable data, every later chart and SQL result becomes weak or misleading.
Exploratory analytics	Use Python to find patterns in traffic, busy days, long trips, active zones, or route demand.	This is where Ashish learns to tell the story behind the dataset before building the app.
SQL layer	Store the cleaned data in a database, write queries, and create summary views for reuse.	Real analytics projects need structured querying, not only notebooks or spreadsheets.
Application layer	Create API endpoints that return totals, trends, summaries, and route tables as JSON.	The backend connects analytics logic to the user interface.
Presentation layer	Build a dashboard with KPI cards, charts, and one or two filters.	The frontend turns analysis into something a user can actually read and interact with.
Cloud basics	Publish the dashboard and understand how hosting works in AWS.	This gives practical support to Ashish's AWS beginner certification preparation.

Suggested folder structure

Create the project with a clean structure from the first day so every file has an obvious home. That habit is important because full-stack projects become messy very quickly when data files, code files, screenshots, and notes are mixed together.

Folder	Purpose
data/raw	Original dataset files kept unchanged.
data/cleaned	Cleaned CSV files that are ready for SQL import.
analysis	Python notebook or scripts used for cleaning and exploratory analysis.
sql	Schema file, import notes, queries, and views.
backend	Flask app, config, and requirements.
frontend	HTML, CSS, JavaScript, and chart code.
aws	Deployment notes, bucket name, EC2 steps, and config notes.
screenshots	Images from notebook, SQL output, dashboard, and deployment.
docs	Short planning notes, API plan, and learning notes.

Three rules Ashish should follow for the whole 21 days

1) Do not jump ahead when the current stage is still unclear. 2) Commit to GitHub regularly with honest commit messages. 3) Keep the project small enough to finish; a clean finished project is much stronger than an advanced but incomplete one.

Day 1 - Understand the project and choose the dataset

Focus of the day

The first day is about orientation, not coding. Ashish should create the project folder, read this handbook once fully, and choose one transportation dataset that is rich enough for analysis but not too huge for a beginner. Good options are taxi trips, ride-share logs, bike-share trips, traffic sensor records, or public transport usage logs. He should save the original file in **data/raw** and never modify that original copy directly. Then he should open the dataset in Excel or a notebook and read each column name slowly. If the dataset has columns such as pickup_time, dropoff_time, trip_distance, fare_amount, passenger_count, pickup_zone, or dropoff_zone, he should write a simple note explaining what each one probably means. If a column is confusing, he should read the dataset description on Kaggle and add his best understanding to a notes file in **docs**.

Why this day is important

This day matters because most beginner mistakes happen when someone starts cleaning or coding before understanding what the data actually represents. A project becomes stronger when the analyst understands what each field means in the real world. For urban mobility, that means thinking about trips, routes, timing, congestion, and demand.

Detailed task checklist

- Create the full project folder structure.
- Download and save one dataset in data/raw.
- Write a simple project objective note in docs/project_overview.txt.
- List all dataset columns and their meaning in plain language.

Expected end-of-day output

Project folders exist, the raw file is safely stored, and Ashish has a notes file that explains the dataset in simple language.

Teacher note / beginner reminder

Do not start cleaning the data on day 1. The goal is understanding, not speed.

Day 2 - Set up GitHub from the beginning

Focus of the day

Today Ashish should create a GitHub repository with a clean professional name such as **smart-urban-mobility-analytics**. He should add a short description, initialize the repository with a README, and upload the project structure. The README does not need to be perfect yet, but it should already mention the project topic, tools, and goal. He should also learn the basic commands or GitHub Desktop workflow needed to make the first commit. The commit message should be meaningful, such as **Initial project structure and dataset notes**. If he is unfamiliar with GitHub, this is the right day to learn the idea of version control: saving progress in stages instead of storing random copies like final2_new_last_reallyfinal.

Why this day is important

This matters because GitHub is part of the project, not an extra decoration at the end. Recruiters and mentors can learn a lot from repository structure, commit history, and documentation discipline. It also protects the project from accidental loss.

Detailed task checklist

- Create the GitHub repository.
- Upload the folder structure and initial notes.
- Add a short README with project title, objective, and tools.
- Make the first commit and push it.

Expected end-of-day output

A live repository exists with the starting structure and first commit.

Teacher note / beginner reminder

Do not wait until the last week to upload everything. The project should grow in GitHub step by step.

Day 3 - Inspect the raw data carefully

Focus of the day

Now Ashish begins real data work. He should open the raw file in Excel or Pandas and inspect the quality of the dataset. He should look for blank cells, strange values, duplicate rows, unexpected data types, impossible timestamps, missing route names, negative distances, or badly formatted text. If the dataset has timestamp columns, he should check whether they are text, full datetime, or inconsistent strings. He should create a cleaning-notes file where every issue is written in one sentence. For example: *trip_duration contains some negative values* or *pickup_time needs conversion to datetime*. He should not fix everything at once; today is mostly an inspection and issue-listing exercise.

Why this day is important

This step is important because analysis quality depends on data quality. Many beginners make charts immediately and then discover later that their results were shaped by missing values, duplicates, or broken timestamps. Careful inspection prevents false confidence.

Detailed task checklist

- Load the raw data and preview rows and columns.
- Check null counts for important fields.
- Inspect timestamp, numeric, and categorical columns.
- Create a cleaning issue log in docs or analysis notes.

Expected end-of-day output

Ashish has a list of all major data problems that must be fixed before analysis.

Teacher note / beginner reminder

Do not guess that the data is clean just because it loaded successfully.

Day 4 - Clean the dataset and standardize columns

Focus of the day

Today is the first active cleaning day. Ashish should create a new cleaned copy of the dataset and begin standardizing it. Column names should be made coding-friendly, for example **pickup_time**, **dropoff_time**, **pickup_zone**, **dropoff_zone**, **trip_distance**, and **fare_amount**. Timestamp fields should be converted properly, not left as raw text. Text categories should be standardized so that values do not appear in multiple forms. For instance, if a mode column contains Taxi, taxi, TAXI, and cab, he should decide what cleaned categories to keep. Duplicate rows should be reviewed and removed if they are clearly true duplicates. Numeric columns should be checked for impossible values such as negative distance or negative fare where that makes no sense.

Why this day is important

Cleaning is important because it creates a trustworthy version of the data that both SQL and the frontend can use later. It also teaches Ashish that analytics work is not just chart-making; it starts with reliable preparation.

Detailed task checklist

- Rename columns into clean snake_case format.
- Convert text dates into proper datetime format.
- Standardize categories and remove or flag impossible values.
- Export the result as a cleaned CSV into data/cleaned.

Expected end-of-day output

A cleaned CSV exists and can be used for analysis and SQL import.

Teacher note / beginner reminder

Keep the raw dataset untouched in data/raw. Cleaning should happen only on a working copy.

Day 5 - Engineer new features for analysis

Focus of the day

Once the data is cleaned, Ashish should create new columns that make traffic analysis easier. If the dataset contains pickup and dropoff timestamps, he can calculate **trip_duration_minutes**. From a timestamp he can derive **hour_of_day**, **day_of_week**, **month**, and even **is_weekend**. If location columns are available, he can create route labels such as pickup_zone + dropoff_zone. If a distance column exists, he can also create distance buckets like short, medium, and long trip. These engineered columns help reveal patterns that would otherwise remain hidden.

Why this day is important

Feature engineering matters because raw columns are often not the exact columns needed for insight. Businesses ask questions like when are peak hours, what days are busiest, and which route types take longest. Those questions usually require derived fields.

Detailed task checklist

- Create time-based features such as hour_of_day and day_of_week.
- Create trip_duration if start and end timestamps exist.
- Create route or zone combination fields if location data supports it.
- Save the updated cleaned dataset again.

Expected end-of-day output

The cleaned file now contains analysis-ready fields that make later SQL and charting easier.

Teacher note / beginner reminder

Name new columns clearly. Future you should understand them immediately.

Day 6 - Do the first exploratory analysis in Python

Focus of the day

Today Ashish begins storytelling with data. Using Pandas, he should calculate simple but meaningful summaries such as trips per hour, trips per day, average trip duration, busiest zones, average distance, and possibly route counts. He should not try to answer every question at once. Instead, he should choose 4 to 6 business questions and compute them one by one. He should write short markdown notes under each result explaining what it means in plain English. For example: *Trip demand rises sharply between 7 AM and 9 AM, which suggests commuting behavior.* This becomes the foundation for both the README and the dashboard later.

Why this day is important

This day is important because it turns data preparation into actual analysis. It also helps Ashish decide what charts and KPIs belong in the final dashboard instead of building visuals blindly.

Detailed task checklist

- Open a notebook or Python script in analysis.
- Compute key summaries using groupby and aggregation.
- Write down 3 to 5 findings in plain business language.
- Save notebook progress and commit to GitHub.

Expected end-of-day output

A first exploratory notebook or script exists with meaningful traffic insights.

Teacher note / beginner reminder

Do not overcomplicate the notebook with too many experiments. Focus on the questions the dashboard will later show.

Day 7 - Create visual analysis charts in Python

Focus of the day

With the first insights ready, Ashish should now create charts. Good beginner-friendly visuals include a line chart of hourly trip volume, a bar chart of trips by weekday, a bar chart of top routes or zones, and a histogram of trip duration. He should label axes clearly and choose titles that explain the meaning of the chart, not just the column name. For example, **Hourly Trip Demand Across the Day** is better than **hour_of_day graph**. After creating the charts, he should save them or take screenshots for later documentation. He should also note which three charts are strongest enough to include in the final web dashboard.

Why this day is important

Visualization matters because charts help detect patterns quickly and prepare Ashish to think like a product builder: which numbers need cards, which patterns need line charts, and which distributions need histograms or bars.

Detailed task checklist

- Build 3 to 5 clear charts from the exploratory analysis.
- Use titles, axis labels, and readable formatting.
- Select the strongest charts for future dashboard use.
- Store screenshots inside screenshots/.

Expected end-of-day output

Ashish has a visual understanding of the data and a shortlist of dashboard-worthy charts.

Teacher note / beginner reminder

Do not add too many chart types. Clear simple charts are better than flashy unreadable ones.

Day 8 - Design the SQL database and schema

Focus of the day

The project now moves from analysis to system building. Ashish should install MySQL or PostgreSQL and create a database. He should design one main table for the cleaned trip dataset. If the data is still beginner-sized, one table is enough. The schema file should define each column with appropriate data types: text for zones or route names, integer or decimal for distance and fare, timestamp for trip time, and integer for engineered time fields when useful. He should save this schema in **sql/schema.sql**. Before writing many queries, he should think carefully about how the cleaned CSV columns map into SQL columns.

Why this day is important

This is important because SQL storage is a core part of analytics engineering. It teaches Ashish how analysis leaves the notebook and moves into a structured environment that supports APIs and applications.

Detailed task checklist

- Install the database locally.
- Create the project database.
- Write a clean schema file for one main trips table.
- Review data types before import.

Expected end-of-day output

The database and schema file exist and are ready for loading data.

Teacher note / beginner reminder

Do not create a complex multi-table design unless the dataset clearly requires it. Simplicity helps completion.

Day 9 - Import cleaned data into SQL and validate it

Focus of the day

Today Ashish loads the cleaned CSV into the SQL table. After import, he must validate instead of assuming success. He should run a row count query, preview a few records, check whether timestamps look correct, confirm that numeric fields did not become text accidentally, and compare the row count with the cleaned CSV. If something looks wrong, such as empty timestamps or rounded numeric values, he should fix the import process and try again. He should keep a short note describing how the import was done, especially if a tool-specific setting was required.

Why this day is important

Validation matters because a successful import is not automatically a correct import. Trustworthy SQL work depends on checking the loaded data carefully.

Detailed task checklist

- Load the cleaned CSV into the trips table.
- Run `SELECT count(*)` and preview queries.
- Check data types and sample values after import.
- Document any issues solved during loading.

Expected end-of-day output

The SQL table is populated correctly and validated against the cleaned file.

Teacher note / beginner reminder

Never move ahead to dashboard logic before confirming that the SQL data matches the cleaned dataset.

Day 10 - Write beginner SQL queries that answer direct questions

Focus of the day

Ashish should now practice foundational SQL. He should write simple SELECT, GROUP BY, ORDER BY, and aggregate queries that answer direct questions such as total trips, trips by day_of_week, average trip duration, average distance, trips by pickup zone, and trips by hour_of_day. He should place each query in **sql/basic_queries.sql** and write a one-line comment above each query explaining what it does. He should run each query and inspect whether the result makes business sense. Where possible, he should compare one or two outputs with Python notebook results to build confidence.

Why this day is important

This step matters because beginner analytics careers rely heavily on SQL. The project should show not only that Ashish can build charts, but also that he can query a database cleanly and interpret results.

Detailed task checklist

- Create a basic_queries.sql file.
- Write clear commented queries for counts, averages, and simple groupings.
- Run each query and check whether the result is logical.
- Save screenshots or notes of a few outputs.

Expected end-of-day output

A tested SQL file exists with the main beginner-level analytics queries.

Teacher note / beginner reminder

Do not combine too many ideas into one huge query. Small clear queries are easier to debug and explain.

Day 11 - Write stronger business-focused SQL analytics

Focus of the day

Now Ashish should go beyond direct counts and create more decision-oriented queries. Examples include peak-hour analysis, busiest weekdays versus weekends, top routes by volume, average trip duration by route or zone, percentage of long trips, and maybe a route delay style measure if the dataset supports it. If zone or route fields are noisy, he can restrict to the top 10 categories so outputs stay readable. This is also a good day to practice CASE WHEN for bucketing durations or distances into short, medium, and long categories.

Why this day is important

This matters because portfolios become more impressive when the project answers practical operational questions instead of only summarizing rows. It shows Ashish can think in terms of decisions and system behavior.

Detailed task checklist

- Create sql/analytics_queries.sql.
- Write peak-hour and route-summary queries.
- Use CASE WHEN to create buckets if helpful.
- Keep outputs readable and business-oriented.

Expected end-of-day output

A second SQL file exists with richer insights that can feed the web dashboard.

Teacher note / beginner reminder

Do not write analytics that the dataset cannot support. Use only questions that match the available fields.

Day 12 - Create SQL views for reuse

Focus of the day

Today Ashish should create reusable SQL views such as **hourly_summary**, **weekday_summary**, **route_summary**, and **zone_summary**. A view is useful because the backend can read a simple summary object instead of repeating long SQL logic again and again. Each view should have a clean name and a clear purpose. He should test them with `SELECT *` and make sure the returned columns are named clearly enough for direct use in JSON responses later.

Why this day is important

Views matter because they simplify full-stack integration. They also teach Ashish a practical pattern used in analytics systems: store detailed data once, summarize it cleanly, and reuse those summaries in applications.

Detailed task checklist

- Create a `views.sql` file.
- Define summary views for the dashboard and API.
- Test each view with `SELECT *`.
- Commit the SQL milestone to GitHub.

Expected end-of-day output

Reusable SQL views exist and are ready for backend integration.

Teacher note / beginner reminder

Choose readable column names in views; unclear names create confusion in the API layer.

Day 13 - Improve GitHub organization and README

Focus of the day

At this point the project already has real substance, so Ashish should pause and improve presentation. He should update the README with the project objective, dataset source, folder structure, tools used, and what has been completed so far in Python and SQL. He should add a short section describing key findings from exploratory analysis. Screenshots of notebook outputs or charts can be added later, but the README should already look professional and understandable. He should also make sure files are named cleanly and that the repository is not full of random temporary files.

Why this day is important

This matters because documentation is part of engineering. A reviewer often reads the README before opening the code. Good documentation makes the project feel intentional and portfolio-ready.

Detailed task checklist

- Update the README with project overview and current progress.
- Clean up file names and remove unnecessary clutter.
- Make separate meaningful commits instead of one messy upload.
- Check that the repository is easy to navigate.

Expected end-of-day output

The GitHub repository now feels organized and professional.

Teacher note / beginner reminder

Do not leave the README blank until the end. It should grow with the project.

Day 14 - Plan the backend before coding it

Focus of the day

Today Ashish should think like a system designer. In **docs/backend_plan.txt**, he should write which endpoints the project actually needs. For example: **/kpis** for total trips and average duration, **/hourly-summary** for line chart data, **/weekday-summary** for day chart data, and **/top-routes** for table or bar chart data. He should write what each endpoint returns and which SQL query or view it will use. He should also write a simple explanation of the roles of frontend, backend, API, and database.

Why this day is important

Planning matters because many beginners open Flask and start coding routes randomly. A small design note reduces confusion and makes the backend easier to implement.

Detailed task checklist

- List the planned endpoints.
- Map each endpoint to a SQL query or view.
- Describe the JSON shape of each endpoint in simple words.
- Decide which endpoints the frontend really needs.

Expected end-of-day output

A small backend plan exists and Ashish understands the architecture before writing code.

Teacher note / beginner reminder

The backend should stay small. Only create endpoints that support the final dashboard.

Day 15 - Set up the Flask backend and database connection

Focus of the day

Now Ashish can create the backend folder properly. He should add files such as **app.py**, **db_config.py**, and **requirements.txt**. He should install Flask and the correct database connector library. The first success target is simple: a local Flask app that runs, has a home route like `/`, and can connect to the SQL database. He does not need to write all analytics endpoints on the first attempt. He should start by proving the server launches and the connection works. If database credentials are sensitive, he can keep them in a config file or environment variables.

Why this day is important

This step is important because backend development becomes much easier once the basic connection is stable. It also teaches a key lesson: build a simple working skeleton first, then add features gradually.

Detailed task checklist

- Create backend files and install dependencies.
- Write a test route that returns a small success message.
- Implement and test the database connection.
- Commit the first working backend version.

Expected end-of-day output

The backend starts locally and successfully talks to the database.

Teacher note / beginner reminder

Do not jump straight into many endpoints before testing the core connection.

Day 16 - Build the first analytics API endpoints

Focus of the day

Today Ashish should create the first working endpoints that return real data as JSON. A strong starting set is one KPI endpoint, one hourly summary endpoint, and one weekday summary endpoint. He should keep route names simple and use consistent JSON structure. For example, a KPI endpoint might return `total_trips`, `average_duration`, and `average_distance`. He should test each endpoint in the browser or Postman and compare the output to SQL results. If the SQL query is already defined in a view, he can read from the view to keep the backend code simpler.

Why this day is important

This matters because the backend is the bridge between SQL and the frontend. Once JSON endpoints work correctly, the dashboard becomes much easier to build.

Detailed task checklist

- Create 3 to 4 starter endpoints.
- Return JSON with clean field names.
- Test endpoints in the browser or Postman.
- Compare endpoint outputs with SQL values.

Expected end-of-day output

Several working API endpoints now serve real analytics data.

Teacher note / beginner reminder

Each endpoint should solve one small problem. Avoid one giant endpoint returning everything.

Day 17 - Add the remaining backend endpoints and harden the API

Focus of the day

Ashish should now add the rest of the routes needed for the dashboard, such as top routes, zone summary, duration buckets, or long-trip percentages. He should also add basic error handling so a failed query returns a readable message instead of a silent crash. If the frontend will later filter by zone or day type, this is also a good stage to think about query parameters, though he should only add simple filters if he can keep them understandable. He should test every route again after changes because new edits can break old behavior.

Why this day is important

The reason this day matters is that complete backend coverage prevents a weak dashboard later. It also teaches Ashish to finish a backend as a coherent layer rather than a collection of random endpoints.

Detailed task checklist

- Add the remaining endpoint set.
- Use SQL views where useful to simplify logic.
- Handle simple errors clearly.
- Retest all endpoints before committing.

Expected end-of-day output

The backend is mostly complete and ready to support the final UI.

Teacher note / beginner reminder

Do not overbuild authentication or advanced architecture. Keep the backend aligned with the learning goal.

Day 18 - Design the frontend layout before filling it with data

Focus of the day

Before connecting data, Ashish should design the visual structure. He can sketch the page on paper or write a simple layout plan. The dashboard should usually include a header, a row of KPI cards, a line chart for hourly traffic, a bar chart for weekday traffic, a chart or table for top routes, and one small insight box or filter area. Then he should create **index.html**, **style.css**, and **script.js**. The page can be static at first with placeholder values.

Why this day is important

This matters because frontend work becomes cleaner when layout decisions are made early. Otherwise many beginners attach data first and only later realize the page has no structure or visual hierarchy.

Detailed task checklist

- Sketch the dashboard sections.
- Create the basic HTML skeleton.
- Style the overall layout with readable spacing.
- Add placeholders for cards, charts, and tables.

Expected end-of-day output

A static but well-structured dashboard layout exists.

Teacher note / beginner reminder

Avoid crowding too many widgets onto one page. Show only the strongest insights.

Day 19 - Connect the frontend with the backend APIs

Focus of the day

Today the dashboard becomes real. Ashish should use JavaScript fetch requests to call the backend endpoints. A good beginner workflow is to first print the returned JSON in the browser console, then map those values into HTML elements one by one. For example, populate the total trip count card, then the average duration card, then move to chart data. If something fails, he should debug calmly: check the endpoint URL, inspect the browser console, confirm CORS settings if needed, and compare the response shape with what the JavaScript expects.

Why this day is important

This step is important because it teaches the core frontend-backend handshake. Once Ashish understands how fetched JSON becomes visible on the page, full-stack work stops feeling mysterious.

Detailed task checklist

- Call the API endpoints with fetch.
- Populate KPI cards first.
- Verify values against backend outputs.
- Debug one endpoint at a time if something breaks.

Expected end-of-day output

The web page now displays live values coming from the backend.

Teacher note / beginner reminder

Do not change many files at once when debugging. Isolate the failing endpoint or DOM element first.

Day 20 - Add charts and one simple interactive filter

Focus of the day

Now Ashish should bring the dashboard to life visually. Using Chart.js, he can add a line chart for hourly demand, a bar chart for weekday demand, and a bar chart or table for top routes. The labels and titles should be easy to read. If he wants one interactive feature, he can add a simple filter such as selecting a route type, zone, or weekday/weekend toggle. The filter should stay small and manageable; it is better to add one filter that works well than several that break the app. He should save polished screenshots after the charts are working.

Why this day is important

This matters because analytics is not only about numbers; it is about communication. A dashboard should help someone understand traffic behavior quickly, not force them to read raw tables.

Detailed task checklist

- Create the main charts using fetched data.
- Label charts clearly and keep the design clean.
- Add one small interactive filter if possible.
- Save dashboard screenshots for the README.

Expected end-of-day output

The dashboard now looks like a real analytics application with visual insights.

Teacher note / beginner reminder

Do not overload the dashboard with too many bright colors or too many chart types.

Day 21 - Prepare AWS hosting and deploy the frontend

Focus of the day

Since Ashish is preparing for an AWS beginner certificate, this day should introduce cloud in a practical but controlled way. He should create an S3 bucket for static website hosting, upload the frontend files, enable static hosting, and note the bucket settings in the **aws** folder. He should understand the concept even if the backend is still running locally: S3 can host static web files such as HTML, CSS, and JavaScript. If the frontend expects local backend URLs, he may temporarily document that the dashboard is partially cloud-hosted while the API remains local. That is still a useful beginner learning outcome.

Why this day is important

This matters because it connects theory to practice. AWS certification preparation becomes easier when Ashish has actually touched services like S3 and understands what static hosting means.

Detailed task checklist

- Create an S3 bucket for website hosting.
- Upload the frontend files and enable hosting.
- Record the deployment steps in `aws/deployment_notes.txt`.
- Test whether the hosted frontend loads correctly.

Expected end-of-day output

The frontend is hosted or at least deployment-ready on S3, and Ashish understands the process.

Teacher note / beginner reminder

Focus on learning the AWS concept clearly rather than chasing a perfect production deployment.

Day 22 - Test end-to-end, refine deployment notes, and prepare final handoff

Focus of the day

The final day is about finishing professionally. Ashish should test the full flow: SQL results should match API results, API results should match dashboard cards and charts, and the repository should contain all essential files. He should update the README with setup steps, screenshots, insights, challenges, and future improvements. He should collect screenshots from Python analysis, SQL outputs, Postman or browser API tests, frontend dashboard, and AWS hosting if available. Finally, he should write a short presentation note so he can explain the project in 3 to 5 minutes: why he chose the dataset, what cleaning he did, what SQL taught him, what the backend does, what the frontend shows, and how AWS fits into the project.

Why this day is important

This step matters because a project is only truly complete when it can be shared, understood, and defended confidently. Final polish often separates a strong portfolio piece from an unfinished practice exercise.

Detailed task checklist

- Run a full system check from data to dashboard.
- Finish the README and add screenshots.
- Write a short explanation note for demo or interview use.
- Make the final GitHub commit and confirm everything is organized.

Expected end-of-day output

The project is submission-ready, portfolio-ready, and explainable in simple language.

Teacher note / beginner reminder

Do not spend the last day adding new features. Use the day to stabilize, document, and present the project well.

Final submission checklist for Ashish

- The cleaned dataset exists in data/cleaned and is clearly separate from the raw file.
- A notebook or Python script contains exploratory analysis and chart logic.
- The SQL folder includes schema.sql, basic_queries.sql, analytics_queries.sql, and views.sql.
- The backend runs locally and the core endpoints return correct JSON.
- The frontend shows KPI cards, charts, and at least one clean summary section or table.
- The GitHub repository has a readable README and regular commit history.
- Screenshots exist for notebook output, SQL results, API testing, and the final dashboard.
- AWS notes exist showing what was learned or deployed through S3 and any deployment experiments.

Mentor grading idea

A simple scoring model can be: 20% data cleaning and exploratory analysis, 20% SQL quality, 15% GitHub discipline and documentation, 20% backend correctness, 15% frontend clarity, and 10% AWS understanding or deployment effort. The most important quality is not complexity; it is whether Ashish can explain the workflow clearly from start to finish.

Final note for Ashish

This project does not need machine learning, authentication, or a complicated architecture. The strongest version is a finished one: clear dataset understanding, clean data, good SQL, a working API, a readable dashboard, and a simple AWS deployment story. A complete workflow done well will teach more and look stronger than an ambitious project left unfinished.